Solution Design: Federal Funds Matchmaker

UNC Solvers (Team 14): Pavan Parola, Advik Jain, Mayank Puri, Aarav Kulkarni · UNC Solve-a-thon 2026 · Version 1.2, 8 November 2026 · Live: https://app-ffm-web.salmonmushroom-cf3974d2.eastus2.azurecontainerapps.io

1. Understand: the user and the problem

Problem. Each North Carolina agency monitors federal opportunities on its own, judges fit against its own priorities, and has to move fast enough to meet deadlines. Opportunities are found late or missed, especially by smaller agencies with little grant capacity. On 7 November 2026 there were **1,460** posted and forecasted federal opportunities on Grants.gov; a keyword search on NC DHHS's own topics returns **1,305** of them, far too many to read.

Users.

- **Primary: an agency's federal-grants coordinator** (we built for NC DHHS and NC DEQ). Reviews a short, ranked list each week, records Pursue / Maybe / Pass, routes each one to the owning division, and sends the list to leadership.
- **Central team (OSBM)**: sees every agency's matches and decisions in one statewide view, spots opportunities that fit more than one agency, and tracks whether federal money is being pursued.
- Division program leads receive routed opportunities; agency leadership uses the funding-by-goal view.

What we learned from the sponsors, and what it changed. The sponsors' main message at Friday's Discovery Session was **scale**: a tool that works for one agency is not enough; it has to extend to every department without a new project each time. That changed our plan in four ways:

- **One shared pull, scored per agency.** Grants.gov is read once a day and every agency is scored from the same data, so adding a department adds no new ingestion.
- **A profile is a file, not code.** Each agency is one JSON profile in a standard template (goals, objectives with plan page numbers, keywords, divisions, USAspending recipient name). The app and the daily job pick up every profile in `agency/profiles/` automatically.
- **We proved it with a second agency.** NC DEQ was added on the same template with no change to the pipeline, and the app gained an agency switcher and a statewide view for OSBM.
- **Cost stays flat.** The fixed cost (about $2–3 a day) is the always-on web app. Each added agency adds only its AI review of the top 45 listings, about 30,000 tokens or under $0.10 a day; listing embeddings are cached and shared across agencies.

What changed between rounds, on the evidence we gathered ourselves:

- **The audience is central, not one agency.** The final brief frames this as an OSBM tool for every agency, so we moved from a one-agency pipeline to one daily pull scored per agency, added NC DEQ on the same template, and built a statewide view.

- **Plans change.** NC DHHS published a new 2026–2030 strategic plan in September 2026. We rebuilt its profile from it during the event (the qualifier had used 2023–2025); matching quality held (section 4).
- **Eligibility is the hard part.** In the qualifier, DHHS was "eligible" for most NIH research calls without being the intended applicant, so research mechanisms and records that exclude state agencies get their own lists.

Requirements we designed to.

1. **Daily**: refreshes itself every morning without anyone pressing a button.
2. **Explainable**: every rank has a published breakdown; every AI claim has a quote checked against the listing.
3. **Human decides**: the system ranks and explains, staff record Pursue / Maybe / Pass.
4. **Works with real plans**: each agency profile comes from its current strategic plan on the OSBM site, with page citations.
5. **Any agency**: one standard profile template, so a new agency is a file, not a project.

2. Architecture

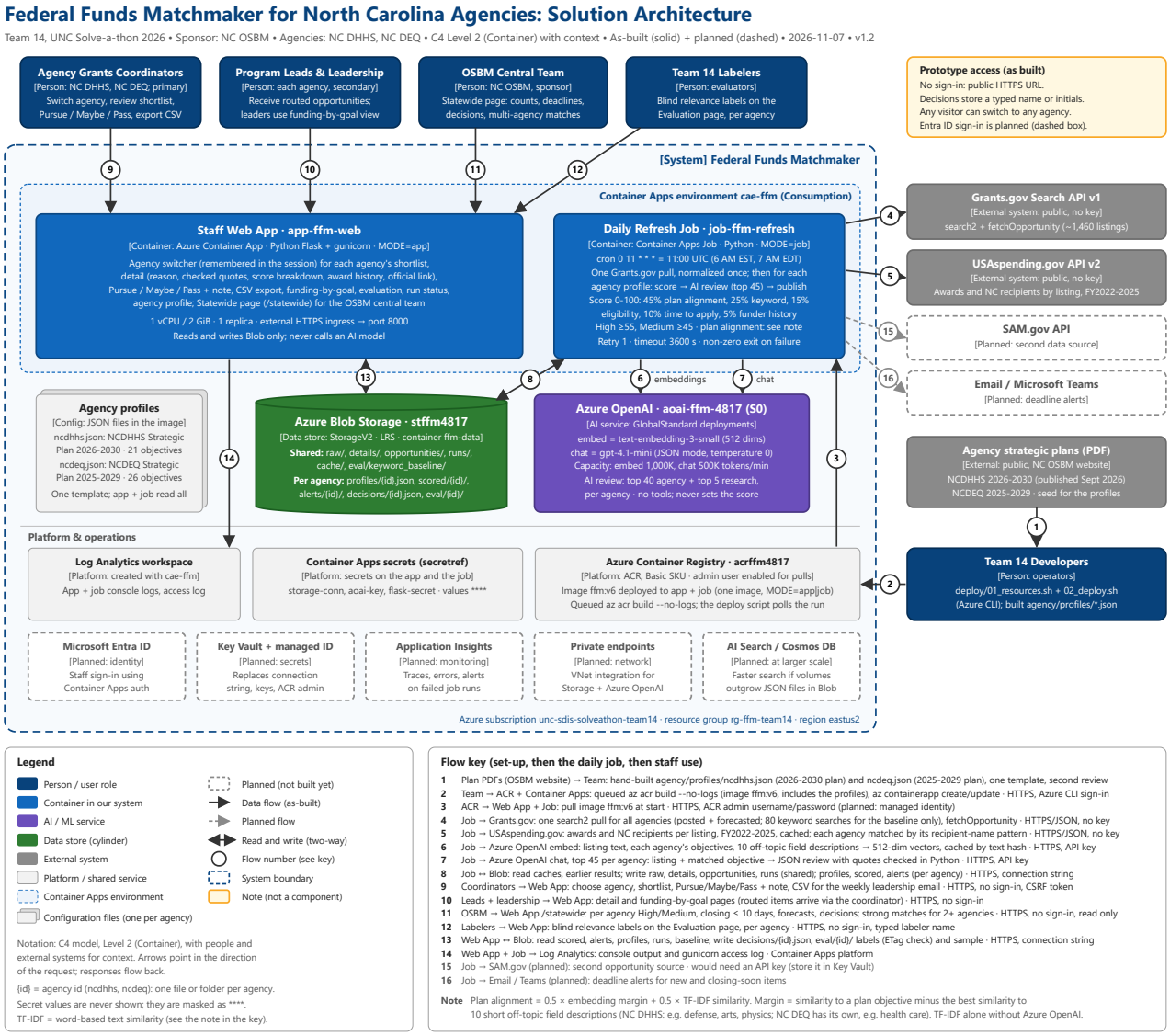


Figure 1. One container image, two roles. The job writes JSON to Blob Storage; the web app reads it and saves staff decisions. Full architecture and threat-model report: `docs/diagrams/ffm-architecture-security/report.html` .

How data flows

1. **Seed (once per agency, and when a plan changes):** our team turned each strategic plan into a profile in one template: NC DHHS 2026–2030 (5 goals, 21 objectives, 4 priority areas) and NC DEQ 2025–2029 (7 goals, 26 objectives), each with page numbers, 40 keywords, program areas, divisions and eligible applicant type. A second teammate reviewed each.
2. **Ingest (daily, 6:00 AM Eastern):** the job calls the Grants.gov API for every posted and forecasted opportunity and its full listing, plus one search per profile keyword (recorded only for the baseline). Raw responses are kept by date for audit and replay.

3. **Normalize:** records are mapped to one schema using the qualifier's tested rules; closed, archived and administrative notices are dropped; duplicates removed by opportunity number; incomplete records flagged.
4. **Score (per agency):** the same opportunities are scored separately against each agency profile: a 0–100 score with a five-part breakdown (section 4).
5. **Summarize:** only each agency's top 45 get an AI summary and "why it matches", with quotes our code verifies.
6. **Serve:** the web app shows each agency's ranked shortlist (agency switcher) and a statewide view; staff decisions are written back to Blob per agency.
7. **Learn:** staff decisions feed back into the next run (similar to a passed or pursued grant moves confidence one step), and the Evaluation page supports blind hand labeling (precision@10 vs a keyword baseline) for the pilot.

3. Azure services

Service	Name	Role	Setting
Resource group	`rg-ffm-team14`	Everything in one place; teardown is one command	East US 2
Blob Storage	`stffm4817` / container `ffm-data`	All state as JSON (data contract below)	Standard_LRS, TLS 1.2, no public blob access
Azure OpenAI	`aoai-ffm-4817`	`embed` = text-embedding-3-small; `chat` = gpt-4.1-mini	GlobalStandard; 1,000K / 500K tokens per minute
Container Registry	`acrffm4817`	One image (`ffm`), built in the cloud	Basic
Container Apps environment	`cae-ffm`	HTTPS ingress, logging (Log Analytics)	Consumption
Container App	`app-ffm-web`	Staff web app (`MODE=app`)	1 vCPU / 2 GiB, 1 replica
Container Apps Job	`job-ffm-refresh`	Daily pipeline (`MODE=job`)	Daily at 11:00 UTC (6 AM EST, 7 AM EDT), 1 retry, 1 h timeout

Cost. About $2–3 a day at pilot scale, mostly the always-on web app; AI costs well under $0.10 a day because embeddings are cached and only 45 summaries are written. There are no databases or clusters to manage.

4. Scoring method

Each signal is between 0 and 1; the score is 100 × the weighted sum. Weights follow the starter kit's recipe.

Signal	Weight	How it is measured
Plan alignment	45%	Average of (a) **embedding margin**: similarity to each plan objective minus similarity to the nearest of ten unrelated fields (defence, arts, physics, …), and (b) **TF-IDF** word similarity. The best objective becomes the matched goal
Keyword / program fit	25%	Profile keywords found; a title hit counts double; full credit at 4

Eligibility	15%	1 if state governments are listed (or the text names state agencies); 0.8 unrestricted; 0.4 unclear; 0 if the text limits applicants to universities, tribes or individuals
Time to apply	10%	Full credit at 60+ days left, none at 0; forecasts 0.7; out-of-date forecasts 0.3
Funder history	5%	USAspending.gov FY2022–25: NC DHHS received awards under this assistance listing (1.0), other state agencies receive ≥25% (0.7), otherwise lower

Confidence: High ≥ 55, Medium ≥ 45, Low below. Low is hidden by default, one click away. **Separate lists:** NIH-style research mechanisms (R01, U01, …) go to a "research (university partner)" list; records whose eligibility excludes state agencies go to "not eligible". Nothing is deleted.

Why this design, with numbers. We tuned the plan-alignment signal on the 1,037 live opportunities that the qualifier's AI pipeline had already reviewed (`src/tune_semantic.py`, NC DHHS 2026–2030 profile). Against whether they are on topic for DHHS: raw embeddings AUC 0.82 (62% of the top 50 on topic), TF-IDF 0.82 (54%), **hybrid 0.90 (76%)**. The full five-signal score reaches **AUC 0.92** against "on topic and DHHS can apply". Raw embeddings put a Navy research announcement among the top matches; the margin against unrelated fields fixed that. Each agency has its own unrelated fields (environmental topics are unrelated for DHHS, and health topics for DEQ). NC DEQ uses the same weights and thresholds untuned.

What the AI may and may not do. The AI never sets the score. It may (a) move a record to "not eligible" only with an eligibility quote our code verified, or (b) lower confidence one step when it says the match is weak. Both are shown on the record.

5. Data contract

All components read and write JSON at fixed paths in `ffm-data` (full field list: `docs/data_contract.md`).

Path	Written by	Read by
`profiles/{agency}.json`	job (from reviewed repo file)	app
`raw/{date}/grants.json`, `details/{id}.json`	job: ingest	job: replay, audit
`opportunities/latest.json`	job: normalize	anyone
`scored/{agency}/latest.json`	job: score + summarize	app
`alerts/{agency}/latest.json`	job	app
`decisions/{agency}.json`	app	app, evaluation
`eval/sample.json`, `eval/labels.json`	app	evaluation
`runs/{timestamp}.json`	job	app (status page)

Minimum opportunity schema: `id, source, number, title, agency, status, open_date, close_date, days_to_close, eligibility, category, award_ceiling, description, url, ingested_at` plus quality flags.

6. Threats and mitigations

The full STRIDE threat model with trust boundaries is in `docs/diagrams/ffm-architecture-security/report.html`. The main threats:

Threat	Severity	Mitigation	Status
Inconsistent data quality (missing dates, placeholder amounts like $999,999,999, HTML in text)	Medium	One schema; the qualifier's tested rules; incomplete records flagged, not dropped; raw data kept	Implemented
False positives (generic announcements look relevant)	Medium	Embedding margin vs unrelated fields; breakdown and matched goal on every record; Low hidden by default	Implemented
False negatives (short or vague listings)	High impact	Low matches kept one click away; keyword-search coverage and recall on hand labels measured	Implemented
AI hallucination (invented facts or quotes)	High	"Not stated" rule; every quote checked word for word and hidden if not found; problems logged; staff decide	Implemented
Prompt injection in third-party listing text	Medium	Model has no tools and cannot act; JSON output validated; it cannot change the score	Implemented
Anyone with the URL can record a decision (no sign-in)	High	Decisions keep name, time and full history	**Gap**: Entra ID sign-in planned
Secrets exposure	Medium	Keys only as Container Apps secrets; `.env` ignored by git and the image build	Implemented; Key Vault + managed identity planned
Web attacks (XSS, CSRF, clickjacking)	Medium	Auto-escaping, CSRF token on every form, strict Content-Security-Policy, X-Frame-Options DENY	Implemented
Grants.gov outage	Medium	Retries; the job refuses to publish if fewer than 50 listings arrive, so yesterday's list stays; replay from saved raw data	Implemented
Runaway AI cost	Low	Only the top 45 summarized; embeddings cached; per-minute capacity limits	Implemented
Uneven agency documentation	Medium	One profile template for every agency; every goal cites a page; second-person review. Agencies with vaguer plans get vaguer matches, which the template makes visible	Implemented (two agencies)
A plan goes out of date	Medium	Profiles name their plan and its date; when an agency publishes a new plan the profile is rebuilt (done for NC DHHS 2026–2030 during the event)	Implemented

7. Pilot and scaling

60-day pilot (run centrally by OSBM): NC DHHS, NC DEQ and one smaller agency with little grant capacity, one coordinator each, weekly review. Coordinators keep their manual check so recall can be measured fairly. Success = hours saved per week, opportunities pursued that would otherwise have been missed, precision at or above target.

Scaling to every department: writing a profile from an agency's strategic plan takes under a day (we rebuilt NC DHHS's from its new 2026–2030 plan during the event). Add the file, run the tests and redeploy;

the next 6 AM run scores every opportunity for that agency and it appears in the agency switcher and the statewide view. One Grants.gov pull serves all agencies, listing embeddings are shared, and each agency adds under $0.10 a day of AI review. At 20+ agencies the job stays one scheduled run; Azure OpenAI capacity (currently 1M embedding tokens a minute) and Blob Storage are far from their limits.

To production: Entra ID sign-in (Container Apps built-in authentication); Key Vault with managed identity for every secret; Application Insights; private endpoints for Storage and Azure OpenAI; a profile per agency in the same template; email or Teams deadline alerts; feedback-tuned weights from Pursue/Pass decisions; Azure AI Search or Cosmos DB if volumes grow. The architecture does not change shape.

GenAI Field Notes: Federal Funds Matchmaker (finals)

UNC Solvers (Team 14): Pavan Parola, Advik Jain, Mayank Puri, Aarav Kulkarni · UNC Solve-a-thon 2026 · NC DHHS and NC DEQ

1. Tools we used

Tool	Where	What for
Azure OpenAI **text-embedding-3-small** (512 dimensions)	In the product, daily job	Meaning-based similarity between each opportunity and every objective of each agency's plan
Azure OpenAI **gpt-4.1-mini** (JSON mode, temperature 0)	In the product, daily job, top 45 per agency only	Summary, why it matches, eligibility notes, deadline, risk, applicant role, two verbatim quotes
Jev and DeepSeek V4 Pro (qualifier)	Offline, reused	Their 1,303 screenings and 283 deep reviews from the qualifier became our reference set for tuning the daily scorer
Claude Code (AI coding assistant)	Development	Wrote and refactored code to our designs, ran the Azure CLI steps, drafted documents we then edited; a second instance reviewed the app for bugs, accessibility and security
Organizers' diagram skills (`architecture-security-diagram`, `stakeholder-map`) run with an AI agent	Documentation	Drafted the C4 architecture, STRIDE threat model and stakeholder map, which we checked against the code

Everything sent to Azure OpenAI is public: Grants.gov listing text and the public strategic plan. No staff decisions, names or labels are ever sent to a model.

2. Key prompts

a. The rationale prompt (in the product, `prompts/rationale.txt`). The parts that mattered most:

> *You never invent facts. If the announcement does not state something, write "not stated". Every quote you give must be copied character-for-character from the ANNOUNCEMENT TEXT … a program checks each quote against the source. You do not decide the score or the ranking.*

It is grounded in the listing text plus the one plan objective the scorer matched, with its page number, so the "why it matches" has to argue for a specific objective and can say "this does not really fit".

b. Reusing the qualifier's rules (to the coding assistant). "Map the Grants.gov `fetchOpportunity` response onto the qualifier's CSV-export columns so the tested Layer 1 rules (deadlines, placeholder amounts, cost share, eligibility signals, research mechanisms) run unchanged on live data." This let 71 existing tests keep guarding the daily job.

c. Tuning against evidence instead of guessing (to the coding assistant). "For the grants that are in both today's live pull and the qualifier's reviewed set, compare raw embedding similarity, an embedding margin against off-topic domains, TF-IDF, and blends. Report AUC against the Jev relevance labels, Spearman against DeepSeek's alignment scores, and precision of the top 50."

d. The diagram skills. We gave the agent the skill file plus a facts sheet of what is built versus planned (e.g. "no Entra ID sign-in yet; secrets are Container Apps secrets, not Key Vault") so the diagrams would not show services we do not run. A second agent then rendered the diagrams and checked every box against the code.

3. Where the AI was wrong, and how we caught it

1. Embeddings ranked unrelated announcements as top matches. On raw similarity, a Navy and Marine Corps research BAA (0.62), a "Countering Violent Extremism" grant (0.61) and a National Park Service cooperative agreement (0.60) scored among the closest matches to the DHHS plan. Long, generic announcements look similar to everything. *How we caught it:* we read the top 25 instead of trusting the numbers. *What we changed:* (a) we also embed ten short descriptions of unrelated fields (defence, arts, physics, agriculture …) and score the **margin**: closer to the DHHS plan than to the nearest unrelated field? (b) we average that with plain TF-IDF word similarity (which is also the fallback if Azure OpenAI is down). Measured on the 1,037 live opportunities the qualifier's pipeline had already reviewed, against whether it judged them on topic for DHHS, using the NC DHHS profile from the current 2026–2030 plan (`python src/tune_semantic.py` , full table in `docs/tuning_report.md`):

Plan-alignment signal	AUC	Share of top 50 on topic
Raw embedding similarity	0.82	62%
TF-IDF only	0.82	54%
Embedding margin vs off-topic fields	0.90	78%
Hybrid (what we ship)	**0.90**	**76%**

The margin is the real fix; TF-IDF keeps the matcher working without AI. The full five-signal score reaches **AUC 0.92** against "on topic and DHHS can apply", with 76% of its top 50 relevant. We first tuned on the 2023–2025 plan; after switching the profile to the 2026–2030 plan the numbers held (hybrid 0.91 → 0.90, full score 0.91 → 0.92). These are AI reference labels from the qualifier, used for tuning; a pilot should add precision@10 on fresh human labels, which the app's Evaluation page is built to collect. NC DEQ uses the same settings untuned: there is no DEQ reference set yet.

A mistake in our own analysis: our first comparison reported lower numbers (0.78 → 0.84, 40% → 68%) because it counted opportunities listed twice in the qualifier data twice and compared capped scores with many ties. Re-running it as a script exposed this; the numbers above are from the corrected script.

2. The model's "verbatim" quotes were not always verbatim. On the first full run, our checker rejected 8 of 90 quotes. Reading each one:

- 5 were our checker being too strict: inside JSON the model swaps `"Domestic"` for `'Domestic'`, and once it quoted the applicant-type list exactly as our prompt displayed it. We fixed the **checker**, not the model.

- The rest were real errors: it **joined two sentences that are not next to each other** into one quote, dropped list numbering ("1. 1 Letter of Commitment" became "1 Letter of Commitment"), returned "N/A" as a quote, and **invented division ids** that are not in our list ("DSCFS" in one run, "DSCFW" twice in the deployed run, apparently blending "DSS" and "DCFW").

How we caught it: every quote is checked against the listing text by code; a failed quote is hidden from staff and logged on the Evaluation page; unknown ids are dropped. An unverified "not eligible" claim is never acted on. In the first deployed run (NC DHHS only) 77 of 90 quotes were verified word for word; in the final 7 November run for both agencies, **162 of 178** were, and every failure is listed on the Evaluation page.

3. Our AI coding assistant introduced a cache bug. Re-running the job showed "0 cached, 1,443 new" embeddings. Three identical re-posted listings were embedded twice but stored once, so the cache's row count no longer matched and it reset itself every run. *Caught by* reading the job log; *fixed* by embedding each unique text once and asserting the cache stays in step.

4. Following a default blindly was slow. The kit's sample deployment command uses capacity 50 (50,000 tokens a minute); the first full embedding pass would have taken about 30 minutes. We raised it to 1,000 for embeddings and 500 for chat (billing is per token, so it costs the same).

5. A false positive our rules missed (NC DEQ). An EPA opportunity open only to Indian tribes and intertribal consortia ranked #11 for DEQ: its text is squarely environmental, and our eligibility rule only catches "tribes only" when the eligibility field says so in set words. The AI review now quotes the eligibility text that excludes state agencies, and a verified quote moves it to the "not eligible" list, where staff can still see it.

6. The AI caught a false positive our score could not, then made one of its own. NC DEQ's #1 match was a RESTORE Act grant that only funds Gulf Coast states. The score cannot see geography, but the AI review's risk line said so. We added an explicit "excludes North Carolina?" answer with its own verified quote, and at first let a verified "yes" move the record off the shortlist. That correctly removed the RESTORE Act grants and Western-states-only WaterSMART programs, but it also hid a DHHS program for **women living with HIV in the South**: the model read HHS's standard clause that territories *may also* apply as a limit. The quote was real, so the checker passed it; the reading was wrong. *What we changed:* a geographic limit now **flags the record and lowers confidence one step**, with the quote shown, instead of removing it. A missed grant costs more than a minute spent passing a bad match, so the human sees it and decides.

7. An independent AI review of our own app found 19 problems. We asked a separate AI reviewer to try to break the web app. It found real issues we had missed, including the detail page being 6,800 px wide on a phone, spreadsheet formulas being injectable through the CSV export, deadlines going stale when a refresh fails, and anonymous decisions. We checked each finding against the code and fixed 18; the 19th (no sign-in) is the first step of a pilot. Regression tests now cover each fix (96 tests in all).

4. Guardrails that stay on

- The score is a published formula computed in Python. The model cannot change it; it can only move a record to "not eligible" with a **verified** eligibility quote, or lower confidence one step when it says the match is weak. Both are shown on the record.

- AI runs only on the top 45 records per agency a day (about 30,000 new tokens per agency on a normal day, well under $0.10).
- If Azure OpenAI is down, the job falls back to TF-IDF and skips summaries; the app says so.
- Staff make every decision.